

WFI-AI-CFI TM Viewer — Run & Diagnostic Reference

A single-page ops manual for the SCC-SW TM Viewer. Copy this into `docs/OPERATIONS.md` in your repo.



Table of Contents

1. [Directory Layout](#)
2. [Preflight Checks](#)
3. [Starting the Stack](#)
4. [Stopping the Stack](#)
5. [Health Verification](#)
6. [Diagnostic Commands](#)
7. [Common Failure Playbook](#)
8. [Database Operations](#)
9. [Redis Operations](#)
10. [Testing the Ingestion Pipeline](#)
11. [Frontend Build & Typecheck](#)
12. [Cleanup & Reset](#)
13. [Port Map](#)
14. [Log Locations](#)



Directory Layout

```
wfi-ai-cfi-tm-viewer/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── api/                # HTTP + WS routers
│   │   ├── tm_service/
│   │   │   ├── ingestion/     # TCP, UDP, file, db sources
│   │   │   ├── inspector/     # CCSDS parse, decode, CRC
│   │   │   ├── storage/       # Redis + Postgres writers
│   │   │   ├── playback/      # DB replay
│   │   │   └── registry.py
│   │   ├── tm_dictionary/     # parameters.yaml, links.yaml
│   │   ├── db/models/         # SQLAlchemy ORM
│   │   ├── schemas/          # Pydantic DTOs
│   │   └── sim/               # local simulator
│   ├── scripts/
│   │   └── send_test_udp.py
│   ├── tests/
│   ├── requirements.txt
│   └── .env
```

```

├── frontend/
│   ├── src/
│   │   ├── main.tsx
│   │   ├── App.tsx
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── store/
│   │   ├── lib/
│   │   └── types/
│   ├── package.json
│   ├── tsconfig.json
│   └── vite.config.ts
└── docker-compose.yml

```

Container Name Convention

This document uses the short names `tm-postgres` and `tm-redis` for readability. On this system:

Document name	Actual container name
<code>tm-postgres</code>	<code>wfi-ai-cfi-tm-viewer-react-postgres-1</code>
<code>tm-redis</code>	<code>wfi-ai-cfi-tm-viewer-react-redis-1</code>

Recommended shell aliases (add to `~/.bashrc`):

```

alias pg='docker exec -it "$(docker ps --filter publish=5432 --format "
{{.Names}}" )" psql -U tm -d tmdb'
alias pgq='docker exec -t "$(docker ps --filter publish=5432 --format "
{{.Names}}" )" psql -U tm -d tmdb -c'
alias redis='docker exec -it "$(docker ps --filter publish=6379 --format "
{{.Names}}" )" redis-cli'

```

Other shortcuts

```

# — WFI-AI-CFI TM Viewer — DB shortcuts —
export PG_CONTAINER=wfi-ai-cfi-tm-viewer-react-postgres-1
export REDIS_CONTAINER=wfi-ai-cfi-tm-viewer-react-redis-1

alias pg='docker exec -it $PG_CONTAINER psql -U tm -d tmdb'
alias pgq='docker exec -t $PG_CONTAINER psql -U tm -d tmdb -c'
alias redis='docker exec -it $REDIS_CONTAINER redis-cli'

```

Reload

```
source ~/.bashrc
```

Now you can run (which is much nicer):

```
# Interactive Postgres shell
pg

# One-shot Postgres query
pgq "SELECT count(*) FROM packets;"
pgq "SELECT pg_size_pretty(pg_database_size('tmdb'));"
pgq "SELECT apid, count(*) FROM packets GROUP BY apid ORDER BY count(*)
DESC LIMIT 5;"

# Redis commands
redis XLEN tm.live
redis INFO memory | grep used_memory_human
redis XREVRANGE tm.live + - COUNT 5
```

✓ Preflight Checks

Run these from a fresh terminal before starting anything.

```
# Toolchain versions
python --version          # expect 3.11+
node --version            # expect 20+
npm --version             # expect 10+
docker --version
docker compose version

# Are the required ports free?
sudo lsof -i :5432 -i :6379 -i :8000 -i :9100 -i :9200 -i :5173

# Is Docker running?
docker info | grep -i "server version"

# Is the conda/venv correct?
which python
which uvicorn
which npm
```

Expected: no port collisions. If a port is bound by a stale process, note the PID; you'll kill it in the [Cleanup section](#).



Starting the Stack

Order matters

1. **Infrastructure** (Postgres, Redis)
2. **Backend** (uvicorn)
3. **Frontend** (Vite)

Terminal 1 — Infrastructure

```
# Recommended: use compose from repo root
cd ~/path/to/wfi-ai-cfi-tm-viewer
docker compose up -d postgres redis

# Or if you don't have docker-compose.yml set up, use raw docker:
docker start tm-postgres tm-redis 2>/dev/null || {
  docker run -d --name tm-postgres \
    -e POSTGRES_USER=tm -e POSTGRES_PASSWORD=tm -e POSTGRES_DB=tmdb \
    -p 5432:5432 postgres:16-alpine
  docker run -d --name tm-redis -p 6379:6379 redis:7-alpine
}

# Wait for Postgres to be ready
until docker exec tm-postgres pg_isready -U tm -d tmdb >/dev/null 2>&1; do
  echo "waiting for postgres..."; sleep 1
done
echo "postgres ready"

# Confirm both are up
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Terminal 2 — Backend

```
cd backend
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1

# First time only:
# pip install -r requirements.txt
# cp .env.example .env

uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Expected startup log (last ~10 lines):

```
INFO app.main - ==> Starting TM Viewer backend
INFO app.main - [1/6] Initializing database ...
INFO app.main - [2/6] Connecting to Redis ...
INFO app.main - [3/6] Creating StorageWriter ...
INFO app.main - [4/6] Creating IngestionManager ...
INFO app.main - [5/6] Starting ingestion sources ...
```

```
INFO app.tm_service.ingestion.manager - Starting source: TCP 0.0.0.0:9100
INFO app.tm_service.ingestion.manager - Starting source: UDP 0.0.0.0:9200
INFO app.tm_service.ingestion.tcp_source - TCP source listening on
0.0.0.0:9100
INFO app.tm_service.ingestion.udp_source - UDP source listening on
0.0.0.0:9200
INFO app.main - [6/6] Starting simulator at 10.0 Hz ...
INFO app.main - ==> Startup complete. Sources: TCP:9100, UDP:9200
INFO:      Application startup complete.
INFO app.tm_service.ingestion.tcp_source - TCP client connected:
('127.0.0.1', 5xxxx)
INFO app.sim.tcp_emitter - Simulator connected to 0.0.0.0:9100
```

Terminal 3 — Frontend

```
cd frontend
# First time only:
# npm install

npm run dev
```

Expected:

```
VITE v5.4.8  ready in 300 ms
→ Local:   http://localhost:5173/
```

Browser

Open **http://localhost:5173**



Stopping the Stack

Graceful stop

```
# Frontend:      Ctrl+C in its terminal
# Backend:       Ctrl+C in its terminal
# Infrastructure:
docker compose stop postgres redis
# or:
docker stop tm-postgres tm-redis
```

Force-stop everything

```

pkill -f 'uvicorn app.main:app'
pkill -f 'vite'
docker stop tm-postgres tm-redis

```

Health Verification

One-shot backend health check

```
curl -s http://localhost:8000/api/meta | python -m json.tool | head -40
```

Expected: JSON with **subsystems**, **links**, **apids**.

Quick counts (3-line sanity)

```

echo "=== APID map size ==="
curl -s http://localhost:8000/api/meta | python -c "import
sys,json;print(len(json.load(sys.stdin)['apids']))"
# expect: 18

echo "=== Redis stream length ==="
docker exec -it tm-redis redis-cli XLEN tm.live
# expect: ~10000 (capped) and stable, or growing if maxlen higher

echo "=== Postgres row count ==="
docker exec -it tm-postgres psql -U tm -d tmdb -t -c "SELECT count(*) FROM
packets;"
# expect: growing over time

```

Full-stack dashboard

```

watch -n 2 '
echo "— Redis tm.live —";
docker exec tm-redis redis-cli XLEN tm.live;
echo;
echo "— Postgres packets —";
docker exec tm-postgres psql -U tm -d tmdb -t -c "SELECT count(*) FROM
packets;";
echo;
echo "— Last 3 packets —";
docker exec tm-postgres psql -U tm -d tmdb -c "SELECT ts, subsystem, card
FROM packets ORDER BY ts DESC LIMIT 3;"
'

```

Press **Ctrl+C** to exit the watch loop.

Diagnostic Commands

Backend — is the process alive?

```
ps aux | grep -E 'uvicorn|vite' | grep -v grep
```

Backend — is it listening?

```
ss -tlnp | grep -E '8000|9100|9200'
# or:
sudo lsof -i :8000 -i :9100 -i :9200
```

Backend — HTTP endpoints

```
# Health / meta
curl -s http://localhost:8000/api/meta | python -m json.tool | head
curl -s http://localhost:8000/api/links | python -m json.tool | head

# Swagger UI (open in browser)
xdg-open http://localhost:8000/docs      # Linux
open http://localhost:8000/docs         # macOS
```

Backend — WebSocket test

The exact channel the frontend uses.

```
cd backend && source .venv/bin/activate
pip install websockets # first time only

python - <<'PY'
import asyncio, json
import websockets

async def main():
    async with websockets.connect("ws://localhost:8000/ws/telemetry") as
ws:
    for _ in range(3):
        msg = json.loads(await ws.recv())
        print(f"frame: {len(msg['packets'])} packets")
        for p in msg['packets'][:2]:
            print(f"  0x{p['apid']:X} {p['subsystem']:8s}
{p['card']:15s} ")
```

```

        f"fields={len(p['fields'])} crc_ok={p['crc_ok']}")

asyncio.run(main())
PY

```

Expected: 3 frames of 3 packets each, all with `crc_ok=True`.

Registry — sanity check

```

cd backend && source .venv/bin/activate

python - <<'PY'
from app.tm_service.registry import get_registry
r = get_registry()
print(f"APIIDs:      {len(r.apid_map)}")
print(f"Subsystems: {len(r.subsystems)}")
print(f"Links:       {len(r.links)}")
for apid, spec in sorted(r.apid_map.items()):
    print(f"  0x{apid:03X} {spec['subsystem']:8s} {spec['card']:20s} "
          f"fields={len(spec['fields'])}")
PY

```

Expected:

```

APIIDs:      18
Subsystems:  6
Links:       7
  0x100  CAMERA  CAM_ELEC          fields=11
  0x110  CDPM_P  CDPM_P_FPGA          fields=9
  ...

```

Decoder — round-trip test

Confirms build → decode → Pydantic all work.

```

cd backend && source .venv/bin/activate

python - <<'PY'
import time
from app.tm_service.sim.spacecraft_model import SimState
from app.tm_service.sim.packet_builder import build_all
from app.tm_service.inspector.decoder import decode
from app.schemas.packet import DecodedPacket

s = SimState(); s.tick()
frames = build_all(s)
print(f"built {len(frames)} frames")

```

```

for f in frames[:1]:
    pkt = DecodedPacket(**decode(f, time.time()))
    print(f"apid=0x{pkt.apid:X} sub={pkt.subsystem} card={pkt.card}")
    for name, fld in list(pkt.fields.items())[:4]:
        print(f"  {name:30s} = {fld.value} {fld.unit} [{fld.status}]")
PY

```

Config — effective settings

```

cd backend && source .venv/bin/activate

python - <<'PY'
from app.config import settings
for k, v in settings.model_dump().items():
    print(f"{k:25s} = {v}")
PY

```

Confirms which `.env` is actually loaded.



Common Failure Playbook

`asyncpg.exceptions.InvalidPasswordError`

Stale Postgres volume from an earlier container.

```

docker rm -f tm-postgres
docker volume ls -q | grep -i postgres | xargs -r docker volume rm
docker run -d --name tm-postgres \
  -e POSTGRES_USER=tm -e POSTGRES_PASSWORD=tm -e POSTGRES_DB=tmdb \
  -p 5432:5432 postgres:16-alpine
sleep 5
docker exec tm-postgres psql -U tm -d tmdb -c "SELECT current_user,
current_database();"

```

Application startup complete but no TCP source listening

Sources aren't being started. Check `backend/app/main.py`:

```

grep -n "start_tcp\|start_udp\|manager" backend/app/main.py | head -20

```

Should show both `await manager.start_tcp(...)` and `await manager.start_udp(...)` before `yield` inside `lifespan`.

Packets flowing in logs but frontend shows nothing

```
# 1) Is the WS actually carrying packets?
cd backend && source .venv/bin/activate
python -c "
import asyncio, json, websockets
async def go():
    async with websockets.connect('ws://localhost:8000/ws/telemetry') as
ws:
        print(len(json.loads(await ws.recv())['packets']), 'packets in
first frame')
asyncio.run(go())
"

# 2) Browser console – hard reload and look for red errors
#     Ctrl+Shift+R in the browser

# 3) Vite terminal – is it printing import errors?
```

Simulator disconnects repeatedly

```
# Look at the backend log tail for the disconnect reason
# Common causes:
# - Port 9100 held by a stale process
sudo lsof -i :9100

# - Simulator started before listener was ready
#     (should be fixed by asyncio.sleep(0.5) in main.py)
```

White screen in browser

```
# Check Vite terminal for import errors – it names the exact file
# In the browser:
#     F12 → Console tab → tick "Preserve log" → Ctrl+Shift+R

# Find empty source files (common cause):
cd frontend
find src -type f \( -name "*.ts" -o -name "*.tsx" -o -name "*.css" \) -
empty
```

TypeScript errors

```
cd frontend
npx tsc --noEmit                # list all
```

```
npx tsc --noEmit --pretty false # compact
```

Redis OOM or stream too big

```
docker exec tm-redis redis-cli MEMORY USAGE tm.live
docker exec tm-redis redis-cli INFO memory | grep used_memory_human

# Trim the stream manually
docker exec tm-redis redis-cli XTRIM tm.live MAXLEN ~ 5000
```

Postgres disk filling

```
docker exec tm-postgres psql -U tm -d tmdb -c \
  "SELECT pg_size_pretty(pg_total_relation_size('packets')) AS
packets_size,
          pg_size_pretty(pg_total_relation_size('parameters')) AS
params_size;"

# Purge older than N hours
docker exec tm-postgres psql -U tm -d tmdb -c \
  "DELETE FROM packets WHERE ts < NOW() - INTERVAL '24 hours';"
```



Database Operations

Connect to Postgres

```
docker exec -it tm-postgres psql -U tm -d tmdb
```

Common queries inside `psql`:

```
-- List tables
\d

-- Row counts
SELECT count(*) FROM packets;
SELECT count(*) FROM parameters;

-- Recent packets
SELECT ts, apid, subsystem, card, crc_ok
FROM packets ORDER BY ts DESC LIMIT 20;

-- Packets per subsystem in the last 5 minutes
SELECT subsystem, count(*)
```

```

FROM packets
WHERE ts > NOW() - INTERVAL '5 minutes'
GROUP BY subsystem ORDER BY count(*) DESC;

-- Find all alarms (params with non-OK status)
SELECT p.ts, pa.subsystem, pa.card, pa.name, pa.value_num, pa.unit,
pa.status
FROM parameters pa
JOIN packets p ON p.id = pa.packet_id
WHERE pa.status <> 'OK'
ORDER BY p.ts DESC LIMIT 50;

-- Table sizes
SELECT pg_size_pretty(pg_total_relation_size('packets')) AS packets,
       pg_size_pretty(pg_total_relation_size('parameters')) AS parameters;

-- Exit
\q

```

Reset the database schema (dev only)

```

docker exec tm-postgres psql -U tm -d tmdb -c "DROP TABLE IF EXISTS
parameters, packets CASCADE;"
# Restart uvicorn – init_db() recreates them

```

Backup / restore

```

# Backup
docker exec tm-postgres pg_dump -U tm tmdb | gzip > tmdb-$(date
+%Y%m%d).sql.gz

# Restore
gunzip -c tmdb-YYYYMMDD.sql.gz | docker exec -i tm-postgres psql -U tm -d
tmdb

```



Redis Operations

CLI access

```

docker exec -it tm-redis redis-cli

```

Common commands inside `redis-cli`:

```

XLEN tm.live           # stream length
XINFO STREAM tm.live   # stream metadata
XRANGE tm.live - + COUNT 5  # oldest 5 entries
XREVRANGE tm.live + - COUNT 5 # newest 5 entries
XTRIM tm.live MAXLEN ~ 5000  # trim to ~5000
DBSIZE                 # total keys
INFO memory            # memory usage
QUIT

```

One-liners

```

# Stream length
docker exec tm-redis redis-cli XLEN tm.live

# Watch it grow
watch -n 1 'docker exec tm-redis redis-cli XLEN tm.live'

# Flush the live stream (dev only – does NOT touch Postgres)
docker exec tm-redis redis-cli DEL tm.live

```



Testing the Ingestion Pipeline

Send N packets over UDP

```

cd backend && source .venv/bin/activate
python -m scripts.send_test_udp --host 127.0.0.1 --port 9200 --count 200 --rate 10

```

Send N packets over TCP

```

python -m scripts.send_test_tcp --host 127.0.0.1 --port 9100 --count 200 --rate 10

```

(If `send_test_tcp.py` doesn't exist, use the UDP variant — the TCP listener accepts the same frames.)

Upload a `.bin` dump for offline inspection

```

curl -X POST http://localhost:8000/api/upload \
  -F "file=@/path/to/dump.bin" \
  | python -m json.tool | head -40

```

Publish a single frame via Python

```
cd backend && source .venv/bin/activate

python - <<'PY'
import socket, time
from app.tm_service.sim.spacecraft_model import SimState
from app.tm_service.sim.packet_builder import build_all

s = SimState(); s.tick()
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
for frame in build_all(s):
    sock.sendto(frame, ("127.0.0.1", 9200))
print(f"sent {len(build_all(s))} frames")
PY
```

Frontend Build & Typecheck

Typecheck only

```
cd frontend
npx tsc --noEmit
npx tsc --noEmit --pretty false
```

Lint (if configured)

```
npm run lint 2>/dev/null || echo "no lint script"
```

Production build

```
cd frontend
npm run build
npm run preview
```

Open **http://localhost:4173** (Vite preview default).

Clear Vite cache

```
cd frontend
rm -rf node_modules/.vite
npm run dev
```

Reinstall node_modules from scratch

```
cd frontend
rm -rf node_modules package-lock.json
npm install
```

Find empty source files

```
cd frontend
find src -type f \( -name "*.ts" -o -name "*.tsx" -o -name "*.css" \) -empty
```



Cleanup & Reset

Kill stray processes

```
pkill -f 'uvicorn app.main:app'
pkill -f 'vite'
sudo lsof -i :8000 -i :9100 -i :9200 -i :5173 | awk 'NR>1 {print $2}' |
xargs -r kill -9
```

Stop containers (keep data)

```
docker stop tm-postgres tm-redis
```

Full reset — Docker volumes

```
docker stop tm-postgres tm-redis
docker rm tm-postgres tm-redis
docker volume ls -q | grep -iE 'postgres|redis' | xargs -r docker volume rm
```

Full reset — Python venv

```
cd backend
deactivate 2>/dev/null || true
rm -rf .venv
python -m venv .venv
source .venv/bin/activate
```

```
pip install --upgrade pip
pip install -r requirements.txt
```

Full reset — Python bytecode cache

```
cd backend
find . -type d -name __pycache__ -exec rm -rf {} +
find . -type f -name "*.pyc" -delete
```

Nuclear option — everything

```
# Kill processes
pkill -f 'uvicorn app.main:app' || true
pkill -f 'vite' || true

# Destroy containers + volumes
docker rm -f tm-postgres tm-redis 2>/dev/null || true
docker volume ls -q | grep -iE 'postgres|redis' | xargs -r docker volume rm

# Destroy frontend cache
cd frontend && rm -rf node_modules/.vite dist && cd ..

# Destroy Python caches
cd backend && find . -type d -name __pycache__ -exec rm -rf {} +
2>/dev/null; cd ..

echo "=== reset complete; restart from 'Starting the Stack' ==="
```

Port Map

Port	Service	Direction	Notes
5432	Postgres	—	Playback DB
6379	Redis	—	Live stream DB
8000	FastAPI	HTTP	REST + /docs
8000	FastAPI	WebSocket	/ws/telemetry , /ws/playback
9100	TCP ingestion	Inbound	Simulator / real TM source
9200	UDP ingestion	Inbound	Test frames / real TM source
5173	Vite dev server	Outbound	Frontend (dev)
4173	Vite preview	Outbound	Frontend (prod build preview)

Log Locations

Component	Where
Backend	stdout of <code>uvicorn</code> terminal (or systemd / docker logs if containerized)
Frontend dev server	stdout of <code>npm run dev</code> terminal
Frontend browser console	Browser DevTools → Console
Postgres	<code>docker logs tm-postgres</code>
Redis	<code>docker logs tm-redis</code>

View backend as JSON (if you add a JSON log handler later)

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 2>&1 | jq -R 'fromjson? // .'.
```

Tail all Docker logs

```
docker compose logs -f postgres redis
```

One-Page Cheat Sheet

For the impatient.

```
# — START —————
docker start tm-postgres tm-redis          # or docker compose up -d
postgres redis
cd backend && source .venv/bin/activate && uvicorn app.main:app --host
0.0.0.0 --port 8000 --reload &
cd frontend && npm run dev &

# — VERIFY —————
curl -s http://localhost:8000/api/meta | python -m json.tool | head -5
docker exec tm-redis redis-cli XLEN tm.live
docker exec tm-postgres psql -U tm -d tmdb -t -c "SELECT count(*) FROM
packets;"
ss -tlnp | grep -E '8000|9100|9200'

# — DIAGNOSE —————
ps aux | grep -E 'uvicorn|vite' | grep -v grep
sudo lsof -i :8000 -i :9100 -i :9200
cd frontend && npx tsc --noEmit
find frontend/src -type f -name "*.tsx" -empty
```

```
# — STOP —  
pkill -f 'uvicorn app.main:app'  
pkill -f vite  
docker stop tm-postgres tm-redis  
  
# — RESET —  
docker rm -f tm-postgres tm-redis  
docker volume ls -q | grep -iE 'postgres|redis' | xargs -r docker volume rm  
rm -rf frontend/node_modules/.vite frontend/dist  
find backend -type d -name __pycache__ -exec rm -rf {} +
```

Quick Decision Tree

Frontend blank?

- | Vite terminal has red import errors → find empty src files
- | Browser console has red React errors → ErrorBoundary shows which component
- | Both clean → check WS frames in browser Network tab

Backend won't start?

- | "InvalidPasswordError" → reset Postgres volume
- | "No module named app" → cd backend first
- | "Address already in use" → kill the port holder

No packets flowing?

- | XLEN tm.live stuck at 0 → check backend log for "Simulator connected"
- | Simulator not connecting → check port 9100 collision
- | Simulator connects but no decode → run the decoder round-trip test

Frontend white / crash?

- | ErrorBoundary shows red → fix the named component
- | TypeScript errors → npx tsc --noEmit
- | Empty .ts/.tsx files → find src -empty

Save this file as `docs/OPERATIONS.md` in your repo. When you're onboarding a new engineer or debugging at 2 AM, this is the one place to look.

Want me to also produce a companion `docs/ARCHITECTURE.md` (component diagram, data flow, extension points)? Say the word.